Na poprzednich zajęciach:

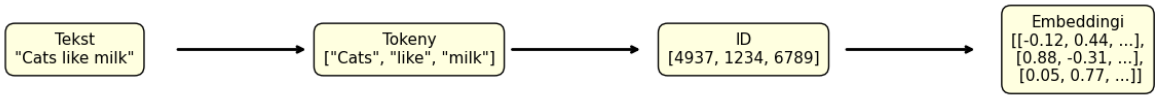
- **tokenizacja** – podział tekstu na słowa / tokeny,
- **spaCy Matcher** – znajdowanie wzorców na podstawie części mowy i cech morfologicznych.

Wektory słów

Tokenizacja a wektory słów

Tokenizacja -> Podziel tekst na kawałki (ID ze słownika)

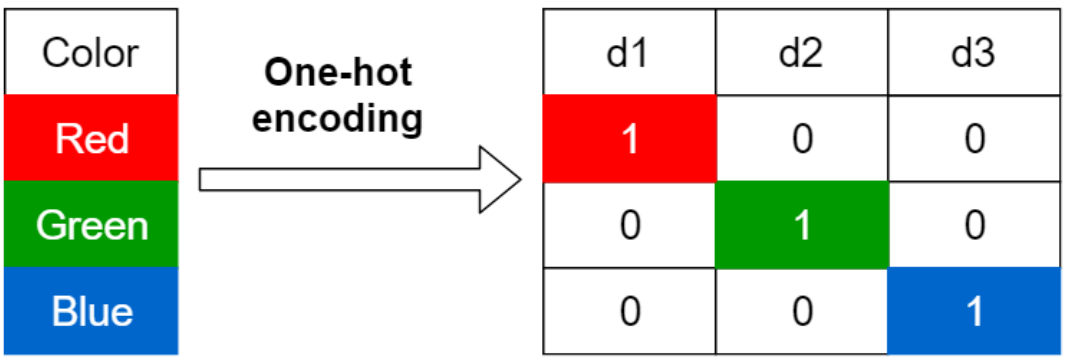
Embedding -> Przedstaw każde słowo w postaci liczb, aby algorytm mógł z nim dalej pracować



Komputer nie „rozumie” tekstu, tylko operuje na liczbach. Zatem każde słowo (np. „król”, „kot”, „miłość”) trzeba jakoś zamienić na liczby, żeby można było nim manipulować w modelu.

Najprostszy pomysł — **one-hot encoding**

```
słownik = [kot, król, królowa]
"król" → [0, 1, 0]
```



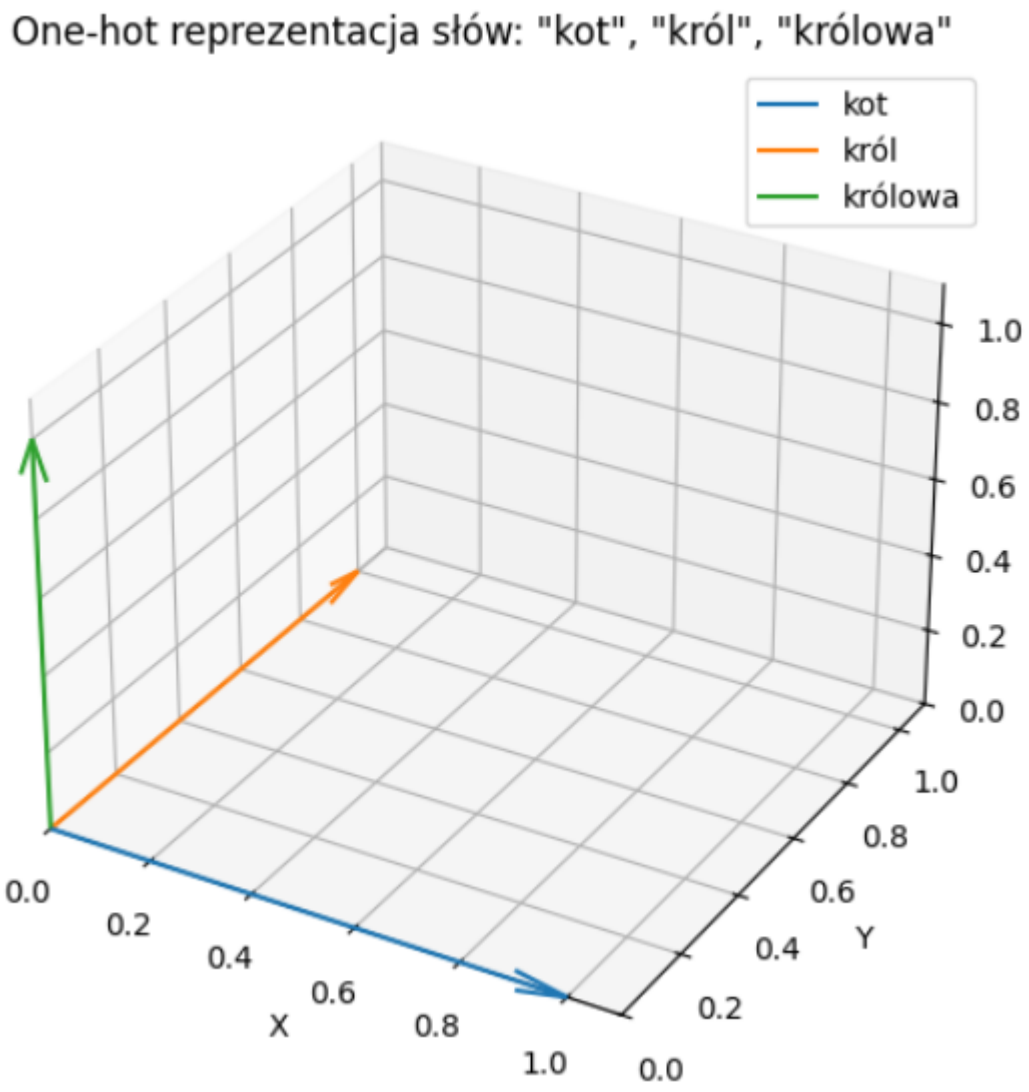
Dlaczego one-hot encoding jest problematyczny?

❌ Bardzo duży wymiar - Jeśli mamy słownik np. 100 000 słów, każdy wektor ma długość 100 000 — to ogromne marnotrawstwo pamięci.

❌ Brak informacji o znaczeniu - Wszystkie wektory są od siebie tak samo odległe — np. “kot” i “pies” są tak samo różne jak “kot” i “samolot”. Nie ma żadnego pojęcia o podobieństwie semantycznym (znaczeniowym).

❌ Brak możliwości uogólniania - Model nie może „zrozumieć”, że pewne słowa mają podobne znaczenie — każde słowo jest całkowicie niezależne.

```
słownik = [kot, król, królowa]
"kot" → [1, 0, 0]
"król" → [0, 1, 0]
"królowa" → [0, 0, 1]
```



Widzimy, że każdy wektor idzie w swoją stronę, każdy jest tak samo odległy od siebie (90 stopni). Żadne dwa nie są do siebie bliżej ani dalej — odległość między każdym z nich jest taka sama.

W naszym przykładzie: kot, król i królowa są tak samo podobni i różni = nie ma wartości semantycznej.

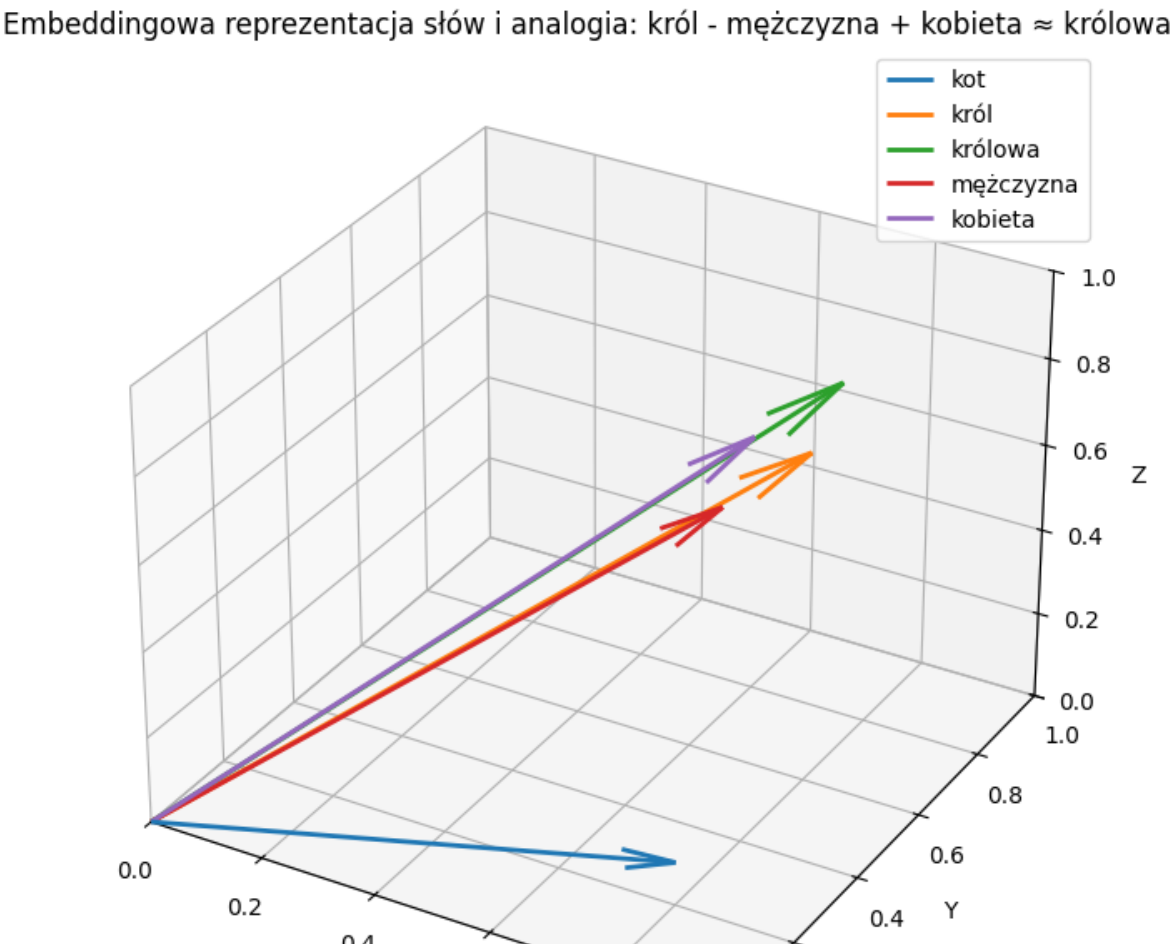
Embedingi

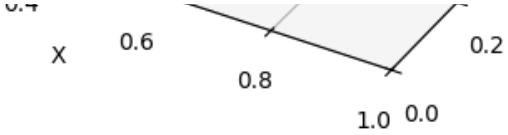
Embeddingi rozwiązują ten problem. W embeddingach uczymy model, żeby słowa, które występują w podobnych kontekstach, miały **podobne wektory**.

Dwa wektory są podobne, kiedy wskazują w podobnym kierunku w przestrzeni wielowymiarowej. Czyli: jeśli narysujesz te wektory jako strzałki zaczynające się w tym samym punkcie (np. w punkcie (0,0,0)) to mają **mały kąt między sobą**.

Przykład:

```
"king" → [0.25, 0.88, -0.12, 0.40, ...]
"queen" → [0.22, 0.90, -0.10, 0.45, ...]
"cat" → [-0.70, 0.11, 0.45, 0.20, ...]
```

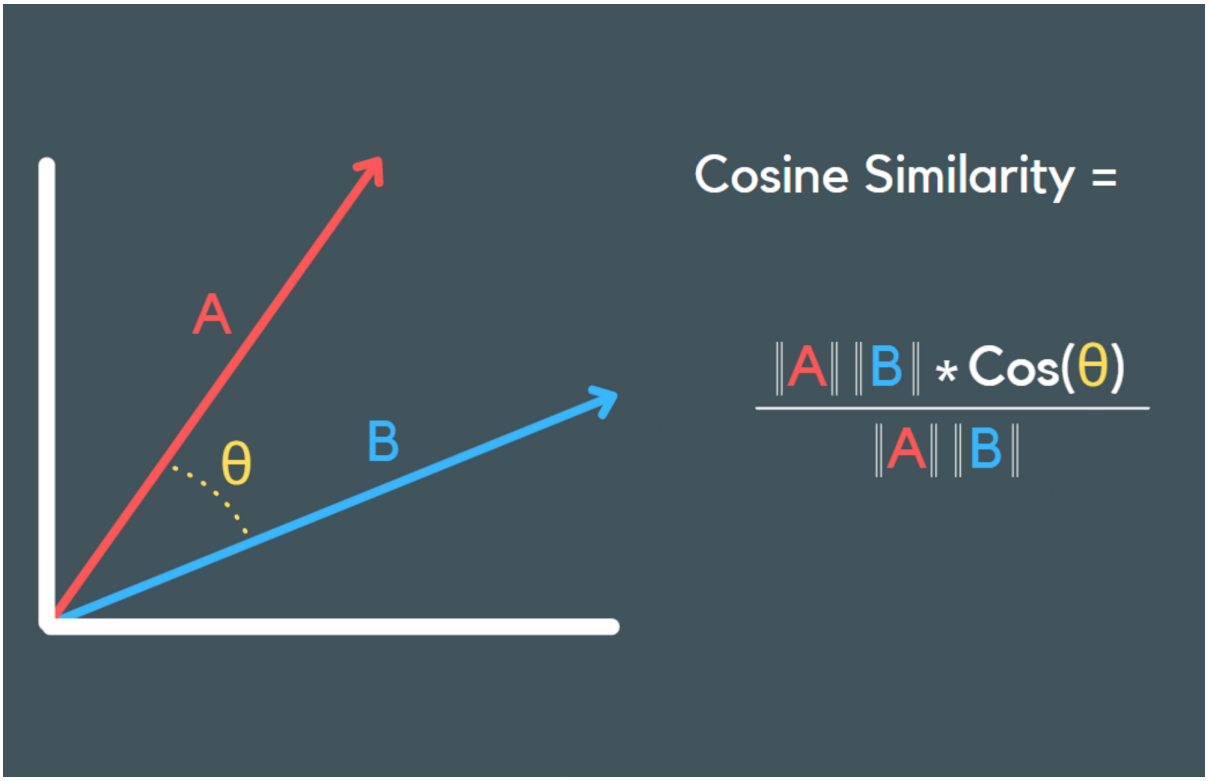





```
król - mężczyzna + kobieta ≈ królowa
```

Teraz punkty w przestrzeni układają się tak, że słowa o podobnym znaczeniu są blisko siebie. To jest właśnie **semantyczne podobieństwo** — podobieństwo w sensie znaczenia, a nie liter czy indeksów w tabeli.

Cosinusowe podobieństwo — jak mierzymy „bliskość znaczeń”



Cosinusowe podobieństwo to sposób na sprawdzenie, czy dwa wektory są „w tym samym kierunku”:

- 1 → bardzo podobne (np. „kot” i „pies”)
- 0 → brak podobieństwa (np. „kot” i „okno”)
- -1 → całkowite przeciwieństwo

Ważne: Patrzymy nie na długość wektorów, tylko na kąt między nimi. Dwa wektory, które „patrzą” w tym samym kierunku, oznaczają słowa o podobnym znaczeniu.

Skąd się biorą te wektory?

Z treningu modelu językowego (np. Word2Vec, GloVe, FastText, BERT). Model uczy się na dużych korpusach tekstu, obserwując konteksty słów.

Efekt:

„king” i „queen” często występują w podobnych zdaniach (np. royal palace, throne, reign) → mają podobne wektory.

„dog” i „cat” też → inna grupa semantyczna.

Trochę praktyki

Zaczniemy od zapoznania się z biblioteką **gensim** Jej nazwa pochodzi od "GENerate SIMilar" — bo jej głównym celem jest znajdowanie podobieństwa między dokumentami lub słowami.

Użyjemy klasy **KeyedVectors** - to strukturą danych w gensim, która przechowuje wektory słów (embeddingi) wytrenowane dowolnym modelem. Czyli po prostu taki kontener do pracy z wektorami - niezależnie od tego, jak one zostały wytrenowane.

```
%pip install gensim
```

```
import gensim
from gensim.models import KeyedVectors
```

weźmiemy sobie polskie pre-trenowane wektory: http://dsmodels.nlp.ipipan.waw.pl pobieramy plik:

```
!wget http://dsmodels.nlp.ipipan.waw.pl/dsmodels/nkjp+wiki-forms-re
```

Rozpakowujemy dane

```
!gzip -d "nkjp+wiki-forms-restricted-300-skipg-ns.txt.gz"
```

Badamy format pliku

```
!head -n 100 nkjp+wiki-forms-restricted-300-skipg-ns.txt
```

```
#Pierwsza linia - lość słów (1982479), rozmiar wektora(300)
#Każda kolejna linia - słowo, liczby reperezentujące wektor
```

(to może chwilę zająć)

```
#Ładowanie wektorów z pliku do naszego KeyedVectors
wv = KeyedVectors.load_word2vec_format('nkjp+wiki-forms-restricted-
```

```
wv.key_to_index.keys()
wv['dom']
```

Znajdziemy najbardziej podobne słowo do słowa kobieta.

```
wv.most_similar('kobieta')
```

Znajdź najbardziej podobne słowo do słowa "kobieta", które nie jest podobne do słowa "mężczyzna"

```
wv.most_similar(positive=['kobieta'], negative=['mężczyzna'])
```

W grupie "pies kot żubr budyń" znajdź słowo, które nie pasuje.

```
wv.doesnt_match(['pies', 'kot', 'żubr', 'budyń'])
```

zbadaj podobieństwo między kotem a psem

```
wv.similarity('kot', 'pies')
```

Teraz zajmiemy się podobieństwami pomiędzy zdaniami

Word mover distance to minimalna suma odległości, jaką trzeba „pokonać”, aby słowa z jednego zdania pokryły znaczeniowo słowa z drugiego zdania.






```
%pip install POT #Po tej instalacji gensim może szybciej i dokładni
```

```
sentence_1 = 'Byłem dzisiaj w kinie'.lower().split()
sentence_2 = 'Poszłam wczoraj do teatru'.lower().split()
```

```
wv.wmdistance(sentence_1, sentence_2)
```

WMD nie jest ograniczone do 0-1, ale przy normalizacji embeddingów zwykle wartości mieszczą się w 0-4 dla krótkich zdań.

wynik ~1.25 sugeruje, że zdania są umiarkowanie podobne semantycznie.

Dla porównania:

- Dwa identyczne zdania → WMD ≈ 0
- Zupełnie różne zdania → WMD > 2.5-3

Flair

Flair to biblioteka NLP od Zalando Research (Python)

Główna idea:

- Klasyczne embeddingi (Word2Vec, GloVe, FastText) dają stały wektor dla każdego słowa, niezależnie od kontekstu:

  "bank" w *"river bank"* i *"money bank"* → ten sam wektor.

- Flair tworzy kontekstowe embeddingi(wektor słowa zależy od jego zdania / kontekstu)

  więc "*bank*" w "*river bank*" będzie inny niż w "*money bank*".

Flair uczy się tego przez modele LSTM działające w dwóch kierunkach (forward i backward).

```
%pip install flair
```

```
from flair.embeddings import FlairEmbeddings, StackedEmbeddings
from flair.data import Sentence
```

```
/home/boro_arch_usr/Documents/Code/PJN/lab_02/.venv/lib/python3.12/s
  from .autonotebook import tqdm as notebook_tqdm
```

```
embeddings = FlairEmbeddings('news-forward')

#tworzymy objekty Sentence
s1 = Sentence("How does it feel to be a scribe?")
s2 = Sentence("There is no such thing as simply good or bad")
sentences = [s1, s2]


# Obliczamy embeddingi(zanurzamy słowa)
embeddings.embed(sentences)

for sentence in sentences: #Zanurzone zdania nadal są obiektem Sent
  print(sentence)
```

```
2026-10-15 13:31:21,475 https://flair.informatik.hu-berlin.de/resour
  0%|          | 0.00/69.7M [00:00<?, ?B/s]
  0%|          | 71.0k/69.7M [00:00<01:51, 656kB/s]
  1%|          | 383k/69.7M [00:00<00:37, 1.93MB/s]
  4%|▍         | 2.85M/69.7M [00:00<00:05, 11.8MB/s]
  8%|▊         | 5.84M/69.7M [00:00<00:03, 19.0MB/s]
 17%|█▋        | 11.8M/69.7M [00:00<00:01, 34.2MB/s]
 24%|██▍       | 16.7M/69.7M [00:00<00:01, 39.6MB/s]
 31%|███       | 21.4M/69.7M [00:00<00:01, 43.0MB/s]
 38%|███▊      | 26.2M/69.7M [00:00<00:01, 45.2MB/s]
 44%|████▍     | 31.0M/69.7M [00:00<00:00, 46.6MB/s]
 51%|█████     | 35.8M/69.7M [00:01<00:00, 47.7MB/s]
 58%|█████▊    | 40.5M/69.7M [00:01<00:00, 48.4MB/s]
 65%|██████▍   | 45.3M/69.7M [00:01<00:00, 48.9MB/s]
 72%|███████▏  | 50.1M/69.7M [00:01<00:00, 49.3MB/s]
 79%|███████▉  | 54.9M/69.7M [00:01<00:00, 49.6MB/s]
 86%|████████▌ | 59.6M/69.7M [00:01<00:00, 49.5MB/s]
 92%|█████████▏| 64.4M/69.7M [00:01<00:00, 49.6MB/s]
 99%|█████████▉| 69.2M/69.7M [00:01<00:00, 49.8MB/s]
100%|██████████| 69.7M/69.7M [00:01<00:00, 41.9MB/s]
2026-10-15 13:31:23,956 copying /tmp/tmp_oyq8git to cache at /home/b
2026-10-15 13:31:23,976 removing temp file /tmp/tmp_oyq8git
Sentence[9]: "How does it feel to be a scribe?"
Sentence[10]: "There is no such thing as simply good or bad"
```

```
#dostajemy embeddingi poszczególnych słów
for sentence in sentences:
  for token in sentence:
    print(token.embedding)
#embeddingi są tensorami
```

```
tensor([-0.0011, -0.0021,  0.0038,  ..., -0.0004,  0.0099,  0.0547],
       device='cuda:0')
tensor([-0.0016, -0.0260,  0.0388,  ...,  0.0031, -0.0070,  0.0025],
       device='cuda:0')
tensor([0.0174, 0.0181, 0.0148,  ..., 0.0051, 0.0051, 0.0130], devic
tensor([ 0.0001,  0.0128,  0.0476,  ..., -0.0084, -0.0023,  0.0106],
       device='cuda:0')
tensor([-2.1672e-03,  8.3752e-05,  2.5102e-02,  ..., -2.8347e-04,
        -1.2090e-02,  2.9513e-02], device='cuda:0')
tensor([-0.0032,  0.0020,  0.0280,  ...,  0.0004, -0.0017, -0.0106],
       device='cuda:0')
tensor([ 5.3283e-03,  3.8049e-05, -2.2042e-02,  ..., -4.1646e-04,
         1.9279e-02, -1.2351e-02], device='cuda:0')
tensor([-1.0594e-03,  4.8670e-05,  2.1462e-02,  ..., -1.1294e-05,
        -4.3308e-03,  3.1280e-03], device='cuda:0')
tensor([ 8.7317e-04,  3.7862e-06, -4.6364e-04,  ..., -1.4222e-04,
         5.4708e-03,  3.0888e-03], device='cuda:0')
tensor([-0.0012, -0.0002, -0.1499,  ...,  0.0024,  0.0017,  0.1648],
       device='cuda:0')
tensor([ 0.0078, -0.0047,  0.0564,  ..., -0.0012,  0.0325,  0.0035],
       device='cuda:0')
tensor([ 5.8073e-02, -7.9190e-05,  9.8895e-02,  ..., -2.7518e-03,
         8.8327e-04,  2.6957e-02], device='cuda:0')
tensor([-0.0004, -0.0036,  0.0781,  ..., -0.0004, -0.0344, -0.0005],
       device='cuda:0')
tensor([ 0.0014,  0.0033, -0.0118,  ..., -0.0005, -0.0164,  0.0293],
       device='cuda:0')
tensor([ 1.2536e-02, -1.0569e-03, -1.1876e-02,  ..., -2.9878e-05,
         4.0988e-03,  2.8744e-02], device='cuda:0')
tensor([-0.0053, -0.0011,  0.0058,  ..., -0.0018,  0.0212,  0.0082],
       device='cuda:0')
tensor([ 2.0060e-04,  9.6346e-04, -2.2168e-02,  ...,  6.8233e-05,
        -1.3982e-01,  3.5051e-03], device='cuda:0')
tensor([-9.0028e-05, -1.7683e-03,  1.0521e-02,  ..., -1.9229e-03,
        -1.2467e-05,  3.2030e-02], device='cuda:0')
tensor([-0.0006,  0.0004, -0.0019,  ..., -0.0005,  0.0061, -0.0073],
       device='cuda:0')
```

```
import numpy as np
```

```
for sentence in sentences:
  for token in sentence:
    print(np.array(token.embedding.to("cpu")))
```

```
[-0.00108573 -0.00213397  0.00377836 ... -0.00035715  0.00994329
  0.05467618]
[-0.00163257 -0.02604987  0.03880595 ...  0.00308893 -0.00699765
  0.0024832 ]
```

```
[0.01740973 0.01807803 0.01477519 ... 0.00507007 0.00508688 0.013028
[ 0.00013419  0.01282794  0.04760637 ... -0.00837355 -0.00231261
  0.01057428]
[-2.1671655e-03  8.3752471e-05  2.5102247e-02 ... -2.8347035e-04
 -1.2090356e-02  2.9513184e-02]
[-0.00319913  0.00195263  0.02804696 ...  0.00040722 -0.00174229
 -0.01062469]
[ 5.3283270e-03  3.8049428e-05 -2.2042062e-02 ... -4.1645818e-04
  1.9278536e-02 -1.2350956e-02]
[-1.0594003e-03  4.8669557e-05  2.1461960e-02 ... -1.1293862e-05
 -4.3307659e-03  3.1279617e-03]
[ 8.7317452e-04  3.7861705e-06 -4.6364017e-04 ... -1.4221962e-04
  5.4707974e-03  3.0888377e-03]
[-0.00118427 -0.00018427 -0.14988413 ...  0.00244022  0.00166776
  0.16482702]
[ 0.00780445 -0.00474086  0.05643403 ... -0.00120921  0.03250856
  0.00350804]
[ 5.8073379e-02 -7.9189973e-05  9.8894514e-02 ... -2.7517765e-03
  8.8326586e-04  2.6957337e-02]
[-0.0003961  -0.00361207  0.07808577 ... -0.00039112 -0.03444915
 -0.00053953]
[ 0.00144947  0.00326685 -0.01180773 ... -0.00050383 -0.01641372
  0.02928467]
[ 1.2535785e-02 -1.0568526e-03 -1.1875718e-02 ... -2.9878380e-05
  4.0988107e-03  2.8744407e-02]
[-0.00529047 -0.00108929  0.00580229 ... -0.00176484  0.02117521
  0.00820001]
[ 2.0059680e-04  9.6345565e-04 -2.2167878e-02 ...  6.8233392e-05
 -1.3981691e-01  3.5051354e-03]
[-9.00283121e-05 -1.76826073e-03  1.05207870e-02 ... -1.92288973e-03
 -1.24667995e-05  3.20298336e-02]
[-0.00058445  0.00037377 -0.00193681 ... -0.00046098  0.00614759
 -0.00726026]
/tmp/ipykernel_25119/3214702032.py:3: DeprecationWarning: __array__
  print(np.array(token.embedding.to("cpu")))
```

Embeddingi dla zdań

```
from flair.embeddings import DocumentPoolEmbeddings
```

```
sentence_embeddings = DocumentPoolEmbeddings([FlairEmbeddings('news
```

```
sentence_embeddings.embed(sentences)
```

```
for sentence in sentences:
  print(sentence.embedding)
```

```
tensor([ 0.0016,  0.0005,  0.0175,  ..., -0.0001,  0.0014,  0.0104],
       device='cuda:0')
tensor([ 0.0073, -0.0008,  0.0052,  ..., -0.0007, -0.0124,  0.0289],
```

```
        device='cuda:0')
```

Zadania

Zadanie1

Załaduj polski model kontekstowych embeddingów Flair:

```
embeddings = FlairEmbeddings('pl-forward')
```

```
2026-10-15 13:32:40,555 https://flair.informatik.hu-berlin.de/resour
  0%|          | 0.00/80.3M [00:00<?, ?B/s]
  0%|          | 71.0k/80.3M [00:00<02:09, 652kB/s]
  0%|          | 360k/80.3M [00:00<00:45, 1.82MB/s]
  3%|▏         | 2.65M/80.3M [00:00<00:07, 10.9MB/s]
  9%|▉         | 7.49M/80.3M [00:00<00:02, 25.7MB/s]
 16%|█▌        | 12.8M/80.3M [00:00<00:01, 36.1MB/s]
 22%|██▏       | 17.6M/80.3M [00:00<00:01, 40.7MB/s]
 28%|██▊       | 22.3M/80.3M [00:00<00:01, 43.4MB/s]
 34%|███▍      | 27.2M/80.3M [00:00<00:01, 45.9MB/s]
 40%|████      | 32.0M/80.3M [00:00<00:01, 47.4MB/s]
 46%|████▌     | 36.8M/80.3M [00:01<00:00, 47.9MB/s]
 52%|█████▏    | 42.1M/80.3M [00:01<00:00, 50.3MB/s]
 59%|█████▉    | 47.1M/80.3M [00:01<00:00, 51.0MB/s]
 65%|██████▌   | 52.0M/80.3M [00:01<00:00, 45.3MB/s]
 70%|███████   | 56.4M/80.3M [00:01<00:00, 38.7MB/s]
 75%|███████▌  | 60.3M/80.3M [00:01<00:00, 30.6MB/s]
 79%|███████▉  | 63.6M/80.3M [00:01<00:00, 27.3MB/s]
 83%|████████▎ | 66.5M/80.3M [00:02<00:00, 24.4MB/s]
 86%|████████▌ | 69.0M/80.3M [00:02<00:00, 21.9MB/s]
 89%|████████▉ | 71.2M/80.3M [00:02<00:00, 19.9MB/s]
 91%|█████████ | 73.2M/80.3M [00:02<00:00, 18.4MB/s]
 93%|█████████▎| 75.0M/80.3M [00:02<00:00, 17.7MB/s]
 96%|█████████▌| 76.8M/80.3M [00:02<00:00, 16.4MB/s]
 98%|█████████▊| 78.3M/80.3M [00:02<00:00, 16.0MB/s]
 99%|█████████▉| 79.9M/80.3M [00:03<00:00, 15.7MB/s]
100%|██████████| 80.3M/80.3M [00:03<00:00, 27.5MB/s]
2026-10-15 13:32:44,348 copying /tmp/tmpx6l8z9j8 to cache at /home/b
2026-10-15 13:32:44,371 removing temp file /tmp/tmpx6l8z9j8
```

```
s1 = Sentence("Zamek w Malborku jest bardzo stary.")
s2 = Sentence("Suwak w kurtce to zamek błyskawiczny")
s3 = Sentence("Sprawdź, czy zamek w drzwiach działa")
sentences = [s1, s2, s3]
```

Oblicz embeddingi dla wszystkich zdań:

```
embeddings.embed(sentences)
```

```
embeddings.embed(sentences)
```

```
[Sentence[7]: "Zamek w Malborku jest bardzo stary.",
 Sentence[6]: "Suwak w kurtce to zamek błyskawiczny",
 Sentence[7]: "Sprawdź, czy zamek w drzwiach działa"]
```

Wypisz każde zdanie i dla słowa "zamek", wypisz pierwsze 5 wymiarów embeddingu:

```
for sentence in sentences:
    print(f"Zdanie: {sentence.to_original_text()}")

    for token in sentence:
        if token.text.lower() == "zamek":
            print(f"Zamek 5 wymiarów: {token.embedding[:5].tolist()
```

```
Zdanie: Zamek w Malborku jest bardzo stary.
Zamek 5 wymiarów: [-0.005450879689306021, -0.021590061485767365, 4.5
Zdanie: Suwak w kurtce to zamek błyskawiczny
Zamek 5 wymiarów: [-0.0043347920291125774, -0.013109492138028145, 0.
Zdanie: Sprawdź, czy zamek w drzwiach działa
Zamek 5 wymiarów: [-0.004302139859646559, -0.02108159102499485, 0.00
```

Czy embeddingi są różne w zależności od znaczenia słowa?

Odpowiedź: Tak

Zadanie 2

Stwórz bota z bazą przykładownych pytań i odpowiedzi, zwracającego odpowiedź na zapytanie na podstawie podobieństwa (WMD albo flair document embeddings) pomiędzy zapytniem użytownika a pytaniami w swojej bazie

Przykład:

Zbiór pytań i odpowiedzi :

```
[
  {"q1" : "a1"},
  {"q2" : "a2"},
  {"q3" : "a3"},
  ...
  {"qn" : "an"},
]
```

Wektoryzujemy pytania:

```
{"q1_vecotorized" : "a1"},
{"q2_vecotorized" : "a2"},
{"q3_vecotorized" : "a3"},
...
{"qn_vecotorized" : "an"},
```

Dla pytania użytkownia pytania q_user wektoryzujemy je i liczmy podobieństwo z pytaniami w bazie

```
sim("q_user_vectorized","q1_vecotorized"), sim("q_user_vectorized","q2_
```

Zwracamy odpowiedź do pytania z bazy najbardziej podobnego do pytania użytkownika

$$argmax(sim(q_{user}vectorized, q_ivecotorized))$$

Do liczenia podobnieństma można użyć np https://docs.scipy.org/doc/scipy/reference/generated/scipy.spatial.distance.cosine.html ale wtedy robimy argmin bo

$$cosine\ distance = 1 - cosine\ similarity$$

```
qa = [
    {
        "q": "W jakich godzinach biblioteka jest otwarta?",
        "a": "Biblioteka jest otwarta od poniedziałku do piątku, od
    },
    {
        "q": "Na jak długo mogę wypożyczyć książkę?",
        "a": "Książkę można wypożyczyć na 30 dni.",
    },
    {
        "q": "Jak założyć konto w bibliotece?",
        "a": "Aby założyć konto, zgłoś się do biblioteki z dokument
    },
    {
        "q": "Ile kosztuje karta biblioteczna?",
        "a": "Wyrobienie karty bibliotecznej jest bezpłatne.",
    },
]
```

```
from flair.embeddings import DocumentPoolEmbeddings

document_embeddings = DocumentPoolEmbeddings(
    [embeddings],
    pooling="mean",
)
```

```
qa_sentences = [Sentence(item["q"]) for item in qa]

document_embeddings.embed(qa_sentences)

qa_vectors = [
    sentence.embedding.detach().cpu().numpy()
    for sentence in qa_sentences
]
```

```
from scipy.spatial.distance import cosine


def calculate_similarity(vec1, vec2):
    return 1 - cosine(vec1, vec2)


def bot(user_question):
    if not user_question.strip():
        raise ValueError("Pytanie nie może być puste.")

    user_sentence = Sentence(user_question)
    document_embeddings.embed(user_sentence)

    user_vector = user_sentence.embedding.detach().cpu().numpy()

    similarities = [
        calculate_similarity(user_vector, question_vector)
        for question_vector in qa_vectors
    ]

    best_index = max(
        range(len(qa)),
        key=lambda i: similarities[i],
    )

    best_match = qa[best_index]

    print(f"Pytanie użytkownika: {user_question}")
    print(f"Najbardziej podobne pytanie z bazy danych: {best_match[
    print(f"Podobieństwo: {similarities[best_index]:.4f}")
    print(f"Odpowiedź: {best_match['a']}")
    print()

    return best_match["a"]
```

```
# TODO - zadaj 3 pytania
test_questions = [
    "O której godzinie biblioteka jest otwarta?",
    "Na ile dni mogę wypożyczyć książkę?",
    "Czy karta biblioteczna jest płatna?",
    "123"
```

```
]

for question in test_questions:
    bot(question)
```

```
Pytanie użytkownika: O której godzinie biblioteka jest otwarta?
Najbardziej podobne pytanie z bazy danych: W jakich godzinach biblio
Podobieństwo: 0.8215
Odpowiedź: Biblioteka jest otwarta od poniedziałku do piątku, od 8:0
Pytanie użytkownika: Na ile dni mogę wypożyczyć książkę?
Najbardziej podobne pytanie z bazy danych: Na jak długo mogę wypożyc
Podobieństwo: 0.8722
Odpowiedź: Książkę można wypożyczyć na 30 dni.
Pytanie użytkownika: Czy karta biblioteczna jest płatna?
Najbardziej podobne pytanie z bazy danych: Ile kosztuje karta biblio
Podobieństwo: 0.7742
Odpowiedź: Wyrobienie karty bibliotecznej jest bezpłatne.
Pytanie użytkownika: 123
Najbardziej podobne pytanie z bazy danych: W jakich godzinach biblio
Podobieństwo: 0.1702
Odpowiedź: Biblioteka jest otwarta od poniedziałku do piątku, od 8:0
```